SHELLMATES CLUB • RAPPORT TECHNIQUE

Discord HR
Role Management Bot

Automatisation et sécurisation de la gestion des rôles Discord à partir de Google Sheets

01 / SYNTHÈSE DU PROJET

Le projet consiste à développer un bot Discord en Python qui synchronise les rôles des membres avec les informations maintenues par l'équipe des ressources humaines dans un Google Sheet. Il vise un serveur de plus de 300 membres et cherche à réduire les opérations manuelles, les oublis et les incohérences d'attribution.

La feuille de calcul constitue la source de référence pour les rôles gérés. À partir de cette source, le bot identifie les membres, compare leur état actuel avec l'état attendu, puis propose ou applique les changements autorisés.

300+ membres visés	**Google Sheets** source de référence	**Python** langage principal	**Slash commands** interface Discord

OBJECTIFS OPÉRATIONNELS

• **Centraliser :** maintenir les affectations de rôles dans une feuille Google Sheets accessible à l'équipe RH.

• **Synchroniser :** ajouter ou retirer uniquement les rôles explicitement placés sous la gestion du bot.

• **Contrôler :** réserver les commandes aux membres RH autorisés et aux administrateurs.

• **Tracer :** produire un résumé d'exécution et conserver un journal des opérations.

Périmètre fonctionnel
Le bot agit uniquement sur les rôles déclarés dans **MANAGED_ROLES**. Les rôles hors périmètre — par exemple Admin ou Modérateur — doivent rester intacts. Cette séparation constitue une règle de sécurité centrale du projet.

02 / CONCEPTION TECHNIQUE

Technologies & fonctionnement

La solution s’appuie sur l’API Discord et l’API Google Sheets. Les bibliothèques retenues permettent de séparer l’accès aux services externes, la logique métier et la configuration.

Technologie	Rôle dans le projet
Python 3.11+	Langage principal et orchestration asynchrone.
discord.py 2.3+	Connexion au serveur Discord, événements et commandes slash.
gspread 6.0+	Lecture des données du Google Sheet.
google-auth 2.20+	Authentification du compte de service Google.
python-dotenv 1.0+	Chargement de la configuration depuis un fichier d’environnement.
pytest 8.0+	Tests automatisés de la logique de synchronisation.

FLUX DE TRAITEMENT

Étape	Traitement
1 — Lecture	Récupérer les lignes du Google Sheet et vérifier les colonnes attendues.
2 — Validation	Normaliser les identifiants, détecter les doublons et analyser les rôles demandés.
3 — Identification	Retrouver le membre Discord par ID si disponible, sinon par username.
4 — Comparaison	Calculer les rôles gérés à ajouter et ceux à retirer.
5 — Exécution	Appliquer les changements si le mode réel est activé et si les permissions le permettent.
6 — Compte rendu	Afficher les statistiques, les erreurs et les éventuels membres ignorés.

Principe de sûreté
La comparaison des rôles doit être indépendante de Discord autant que possible. Elle peut ainsi être testée avec des données simulées avant toute opération sur un serveur réel.

Fonctionnalités principales

Lecture et interprétation de la feuille

Les colonnes attendues sont **DISCORD USERNAME** et **ROLE**. La colonne **DISCORD ID** est optionnelle et, lorsqu'elle est renseignée, devient le moyen d'identification prioritaire. Le bot accepte plusieurs rôles dans une même cellule, séparés par des virgules.

La comparaison des usernames ignore la casse, les espaces superflus et un éventuel caractère @ au début. L'identifiant Discord reste préférable, car un username peut changer.

Synchronisation des rôles

La commande **/sync-roles** compare les rôles actuellement détenus par chaque membre avec ceux définis dans la feuille. Elle ajoute les rôles manquants et retire les rôles gérés qui ne sont plus attendus. Les rôles non gérés sont préservés.

Prévisualisation et vérification

/preview-sync et le paramètre **dry_run:true** permettent de présenter les changements envisagés sans les appliquer. La commande **/check-member** compare la situation d'un membre avec sa ligne dans la feuille.

Compte rendu

Après une exécution, le bot présente le nombre de membres traités, les rôles ajoutés et supprimés, les membres ignorés, les erreurs et la durée d'exécution. Lorsque le détail est trop long pour une réponse Discord, un fichier texte peut être joint.

FONCTIONNALITÉS COMPLÉMENTAIRES DÉCRITES DANS LE RAPPORT FOURNI

Fonctionnalité	Comportement prévu / décrit
Synchronisation planifiée	Tâche périodique configurable, désactivée par défaut.
Attribution à l'arrivée	Événement on_member_join pour traiter un nouveau membre.
Verrou de synchronisation	asyncio.Lock pour empêcher deux synchronisations simultanées dans le même processus.
Gestion des délais	Pause configurable entre opérations afin de limiter les risques de rate limit.
Journal d'audit	Fichier logs/sync.log avec horodatage et informations sur les opérations ; rotation annoncée à 5 Mo.
Déploiement et CI	Dockerfile, docker-compose.yml et workflow GitHub Actions mentionnés.

À distinguer
Les éléments ci-dessus reprennent les fonctionnalités déclarées dans le rapport fourni. Leur présence dans le document ne constitue pas, à elle seule, une preuve de tests réussis ou de disponibilité en production.

Comportements attendus

Situation rencontrée	Comportement défini
Username introuvable	Ignorer la ligne et signaler le membre ; ne pas deviner son identité.
Discord ID renseigné	Utiliser l'ID en priorité ; ne pas basculer silencieusement vers le username si l'ID ne correspond pas.
Lignes en double	Ignorer toutes les lignes ambiguës concernées.
Rôle inexistant sur Discord	Ignorer ce rôle et signaler l'erreur ; les autres rôles valides de la ligne peuvent être appliqués.
Rôle absent de MANAGED_ROLES	Ne jamais l'ajouter ni le supprimer.
Tous les rôles demandés invalides	Ignorer la ligne sans interpréter la situation comme une demande de suppression.
Cellule ROLE vide sur une ligne valide	Retirer tous les rôles gérés du membre ; conserver les rôles hors périmètre.
Permission Manage Roles absente	Avertir en prévisualisation ; annuler la synchronisation réelle selon la règle décrite.
Rôle cible au-dessus du rôle du bot	Bloquer l'opération concernée et rapporter l'erreur.

MESURES DE SÉCURITÉ

Contrôle	Mesure
Contrôle d'accès	Commandes réservées aux membres ayant un rôle RH déclaré dans HR_ROLE_IDS ou aux administrateurs du serveur.
Protection des secrets	Token Discord et identifiants Google conservés hors du dépôt, dans .env et le fichier JSON du compte de service.
Confidentialité	Réponses éphémères pour les commandes afin de limiter l'exposition des détails membres.
Journalisation	Journal d'audit avec horodatage, utilisateur initiateur, action et rôle concerné ; les secrets doivent être masqués.
Moindre privilège	Le compte de service Google reçoit un accès Viewer si la lecture seule suffit ; le bot ne gère que la liste blanche configurée.

Point d'attention
Une cellule ROLE vide n'a pas le même sens qu'une cellule contenant uniquement des rôles invalides. La première représente explicitement une demande de retrait des rôles gérés ; la seconde doit être ignorée pour éviter une suppression accidentelle.

Organisation du code

```
discord_bot_shellmates/
■■■ bot/
■   ■■■ main.py              # démarrage, intents, événements et tâches
■   ■■■ commands.py          # commandes slash
■   ■■■ sheets.py            # lecture, parsing et validation Google Sheets
■   ■■■ sync.py              # comparaison et orchestration des changements
■   ■■■ discord_gateway.py   # accès aux membres et rôles Discord
■   ■■■ permissions.py       # contrôles HR / administrateur
■   ■■■ reporting.py         # résumé et rapports détaillés
■   ■■■ config.py            # chargement et validation de configuration
■   ■■■ logging_config.py    # journalisation et masquage des secrets
■■■ tests/                   # tests pytest
■■■ sample_data/             # données d'exemple
■■■ credentials/             # clé du compte de service (non versionnée)
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
```

CONFIGURATION ET PRÉREQUIS

Élément	Configuration attendue
Discord	Activer Server Members Intent dans le Developer Portal et dans le code ; accorder Manage Roles ; placer le rôle du bot au-dessus des rôles qu'il doit gérer.
Google Cloud	Activer Google Sheets API ; créer un compte de service ; partager la feuille avec son adresse e-mail en lecture seule si aucune écriture n'est nécessaire.
Variables clés	DISCORD_BOT_TOKEN, DISCORD_GUILD_ID, GOOGLE_SHEET_ID, GOOGLE_SERVICE_ACCOUNT_FILE, MANAGED_ROLES, HR_ROLE_IDS, ENABLE_SCHEDULED_SYNC et ENABLE_JOIN_SYNC.
Dépôt Git	Exclure .env, les clés JSON et tout secret du dépôt à l'aide de .gitignore ; ne jamais publier les identifiants dans l'historique Git.

VALIDATION AVANT MISE EN PRODUCTION

■ Vérifier les cas d'ajout, de retrait et de conservation des rôles non gérés.

■ Tester le matching par username (casse, espaces, @) et par Discord ID.

■ Tester les doublons, membres absents, rôles inexistants et cellules ROLE vides.

■ Vérifier qu'une erreur d'accès à Google Sheets n'entraîne aucune modification Discord.

■ Tester les permissions, la hiérarchie des rôles, les réponses éphémères et les limites de débit.

■ Exécuter d'abord une prévisualisation, puis une synchronisation sur un serveur de test.

CONCLUSION

Le projet propose une approche structurée pour automatiser la gestion des rôles d'un serveur Discord à partir d'une source RH centralisée. Les choix de conception — liste blanche des rôles, prévisualisation, contrôles d'accès, journalisation et gestion explicite des cas ambigus — réduisent les risques associés à une synchronisation automatisée.

Le rapport initial qualifie le bot de complet et fonctionnel. Cette version reformule cette conclusion de manière plus vérifiable : la solution est décrite comme implémentée, mais son aptitude à la production doit être confirmée par l'exécution des tests, une recette sur serveur de test et la vérification des paramètres de sécurité.

Document restructuré à partir du rapport de projet et de la spécification technique Shellmates fournis. Les fonctionnalités et paramètres sont repris de ces sources ; les recommandations de validation sont des précautions de mise en production.